

Self Join in DBMS

COMPREHENSIVE EXAM-READY REVISION NOTES

1 What is a Self Join?

A **Self Join** is a join operation in which a table is joined with itself. In other words, the same table is treated as if it were two separate tables by using table aliases.

★ EXAM-READY DEFINITION

Self Join is a join operation in which a relation is joined with itself using different aliases. It is particularly useful when we need to compare rows of the same table with other rows within that same table.

2 Example Scenario: Student–Course Relationship

Consider the relationship table named **STUDY** :

S_ID (Student ID)	C_ID (Course ID)	Since (Start Year)
S1	C1	2016
S2	C2	2017
S1	C2	2017

Primary Key: (**S_ID, C_ID**) — A composite primary key because two attributes together uniquely identify a row.

3 Problem Statement

Problem: Find the Student ID (**S_ID**) of students who are enrolled in *at least two courses*.

"At least two" implies: **Minimum 2 courses (2, 3, 4, ...)**

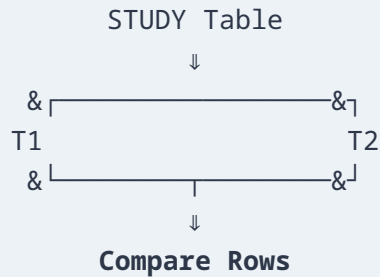
Analysis of Table Data:

- **S1** → Enrolled in **C1** and **C2** (Total = 2 courses) ✓
- **S2** → Enrolled in **C2** only (Total = 1 course) ✗

Expected Answer: **S1**

4 Why Do We Need a Self Join?

Normally, a join combines two or more different tables. However, here all the required data exists inside a single table (`STUDY`). To solve the problem, we need to compare **one row of `STUDY` with another row of `STUDY`** .



5 The Necessity of Table Aliases

We cannot write `FROM STUDY, STUDY` because the DBMS cannot resolve column references. Creating explicitly named aliases solves this:

```
FROM STUDY AS T1, STUDY AS T2
```

- `T1` → First reference to `STUDY`
- `T2` → Second reference to `STUDY`

★ IMPORTANT NOTE

An alias does **not** create a new physical or permanent table in memory/storage. It merely provides a temporary reference name for the table during query execution.

6 Conceptual Intermediate: Cartesian Product

When executing `FROM STUDY AS T1, STUDY AS T2` , the comma performs a Cartesian Product (Cross Join). If `STUDY` contains 3 rows:

$$3 \times 3 = 9 \text{ ext{ rows in intermediate result}}$$

This follows the foundational relational algebra rule: *Join = Cartesian Product + Selection Condition*.

7 Defining the Join Conditions

To identify students taking at least two different courses, two logical conditions must hold simultaneously:

1. **Condition 1 (Same Student):** $T1.S_ID = T2.S_ID$
2. **Condition 2 (Different Courses):** $T1.C_ID \neq T2.C_ID$

 **Golden Rule: Same Student + Different Course = Enrolled in ≥ 2 Courses**

8 Complete SQL Query

```
SELECT T1.S_ID
FROM STUDY AS T1, STUDY AS T2
WHERE T1.S_ID = T2.S_ID
      AND T1.C_ID <> T2.C_ID;
```

9 Step-by-Step Row Comparison Evaluation

Row Pair	T1 Row	T2 Row	T1.S_ID = T2.S_ID	T1.C_ID $\neq$ T2.C_ID	Status
Pair 1	(S1, C1)	(S1, C1)	S1 = S1 TRUE	C1 $\neq$ C1 FALSE	REJECT
Pair 2	(S1, C1)	(S1, C2)	S1 = S1 TRUE	C1 $\neq$ C2 TRUE	SELECT
Pair 3	(S2, C2)	(S1, C2)	S2 = S1 FALSE	—	REJECT

10 Final Result & Qualification

The matching pairs generated from the comparison are:

T1.S_ID	T1.C_ID	T2.S_ID	T2.C_ID
S1	C1	S1	C2
S1	C2	S1	C1

11 Why Qualify Columns (e.g., `T1.S_ID`)?

Writing simply `SELECT S_ID` causes an **Ambiguous Column Reference** error because the system cannot determine whether you mean `T1.S_ID` or `T2.S_ID`.

12 Handling Duplicate Output with `DISTINCT`

Because both pair combinations `(S1, C1) - (S1, C2)` and `(S1, C2) - (S1, C1)` meet the query criteria, `S1` would be printed twice. Use `DISTINCT` to enforce uniqueness:

```
SELECT DISTINCT T1.S_ID
FROM STUDY AS T1, STUDY AS T2
WHERE T1.S_ID = T2.S_ID
      AND T1.C_ID <> T2.C_ID;
```

13 Modern ANSI SQL JOIN Syntax

```
SELECT DISTINCT T1.S_ID
FROM STUDY T1
JOIN STUDY T2
  ON T1.S_ID = T2.S_ID
 AND T1.C_ID <> T2.C_ID;
```

14 Self Join vs. Normal Join

Feature	Normal Join	Self Join
Tables Involved	Joins two or more distinct tables.	Joins a single table with itself.
Example	Employee + Department	STUDY + STUDY
Table Aliases	Optional (useful for readability).	Mandatory (to resolve ambiguity).

15 Classic Real-World Example: Employee–Manager

Consider an **Employee** table containing hierarchical relationships:

Emp_ID	Emp_Name	Manager_ID
E1	Ravi	E3
E2	Amit	E3
E3	Raj	NULL

Query to find Employee and their Manager Name:

```
SELECT E.Emp_Name AS Employee, M.Emp_Name AS Manager
FROM Employee E
JOIN Employee M
  ON E.Manager_ID = M.Emp_ID;
```

! Common Exam Traps & Warnings

△ EXAM TRAP 1: PHYSICAL TABLES

Self Join does **NOT** create duplicate physical tables. `T1` and `T2` are temporary references to the exact same table.

△ EXAM TRAP 2: INCOMPLETE CONDITIONS

`T1.S_ID = T2.S_ID` alone is insufficient! Without `T1.C_ID ≠ T2.C_ID`, every row will match with itself, producing incorrect results.

△ EXAM TRAP 3: "AT LEAST 2" INTERPRETATION

"At least 2" means ≥ 2 (2 or more). Do not confuse it with "exactly 2".

✓ Quick Revision Summary

- **Self Join:** Join a table with itself using mandatory aliases.
- **Core Condition for "At least 2 courses":** `SAME S_ID, DIFFERENT C_ID`
- **Key Formula:** $T1.S_ID = T2.S_ID \text{ AND } T1.C_ID \neq T2.C_ID$
- **Always use** `DISTINCT` to eliminate redundant permutation output.